Final Project — Java Object-Oriented Programming

1. Project Objective

For this final project, you will individually design and develop a **Java application based on Object-Oriented Programming principles**.

You are free to choose the subject of your application.

The objective is not to create the most complex application possible, but to demonstrate that you can **design, structure, test and maintain a Java application using the concepts learned during this course**.

Your application must contain meaningful business logic. It should not simply store and display data.

Project return date : 15/11/26 23:59

Possible project ideas

You are completely free on your application business subject.

You may develop, for example:

- 🎮 A video game / RPG / battle simulator
- 🏦 A personal finance management application
- 📚 A library management system
- 🏥 A hospital management system
- 🏨 A hotel reservation system
- ✈️ A flight or travel management system
- 🛒 An e-commerce / shopping cart application
- …

You are encouraged to choose a subject that interests you.

---

2. Technical Requirements

Your application must be developed in **Java**.

The application must demonstrate the following concepts studied during the course.

2.1 Classes and Objects

Your application must contain several classes representing the concepts of your domain.

You must demonstrate:

- Classes
- Objects
- Attributes
- Methods
- Constructors
- Encapsulation
- Appropriate access modifiers

Your classes should have clear responsibilities.

2.2 Inheritance

Your project must contain at least one meaningful inheritance relationship.

The inheritance relationship must make sense for your application's domain.

Do not create inheritance simply to satisfy the requirement.

2.3 Polymorphism

Your application must demonstrate **polymorphism**.

For example, your application could manipulate different subclasses through a common parent type:

```
List<Character> characters;

characters.add(new Warrior());
characters.add(new Mage());

for (Character character : characters) {
    character.performAction();
}
```

The behavior should depend on the actual object type.

2.4 Abstract Class and/or Interface

Your project must use at least:

- **one abstract class**, and/or
- **one interface**

Both must have a meaningful purpose in your design.

For example:

```
abstract class Character {
    public abstract void attack();
}
```

and:

```
interface Healable {
    void heal();
}
```

You should be able to explain why you chose an abstract class or an interface.

---

3. Business Logic

Your application must implement meaningful **business logic**.

The program should not only create objects and print them.

Examples of business operations include:

- Calculate the total price of an order
- Apply discounts
- Book or cancel a reservation
- Transfer money between accounts
- Determine the winner of a game
- Calculate a player's score
- Check whether a product is available
- Assign students to courses
- Calculate a tournament ranking
- Manage character damage and health
- Detect conflicts between reservations

Your application should contain several operations that require your classes to interact with each other.

---

4. File Loading

When the application starts, it must **load data from a file by default**.

You may choose the file format:

- CSV
- JSON
- TXT
- or another simple format approved by the teacher.

For example, a game could load its characters from:

characters.csv

A library application could load books:

books.csv

A banking application could load accounts:

accounts.csv

The application should use this data to initialize its objects.

Important

The application must not rely exclusively on hard-coded objects such as:

```
new Book(...);
new Book(...);
new Book(...);
```

The initial data must come from the file.

---

5. Exception Handling

Your application must correctly handle possible errors.

You must use:

- `try / catch`
- appropriate standard exceptions
- at least **one custom exception**

You should add proper validation to your business logic and throw exceptions on business errors.

You should avoid catching exceptions without handling them meaningfully.

---

6. Unit Testing

Your project must contain **unit tests**.

Use **JUnit**.

You must test important parts of your business logic.

Your tests should cover:

- Normal situations
- Edge cases
- Invalid situations / exceptions

For example:

```
@Test
void shouldApplyDiscount() {
    ...
}
```

and:

```
@Test
void shouldThrowExceptionWhenBalanceIsInsufficient() {
    ...
}
```

Focus on testing your **business logic**, not simply testing getters and setters.

---

7. SOLID Principles

Your project must demonstrate that you understand and apply the **SOLID principles**.

You are not required to apply every principle everywhere.

However, your architecture should demonstrate meaningful use of principles such as:

- **Single Responsibility Principle**
- **Open/Closed Principle**
- **Liskov Substitution Principle**
- **Interface Segregation Principle**
- **Dependency Inversion Principle**

Your code should avoid large "God classes" responsible for everything.

For example, avoid creating:

Application.java

containing all:

- file loading
- business logic
- user interaction
- validation
- calculations
- object creation
- exception handling

Instead, separate responsibilities into appropriate classes.

---

8. Collections and Java Features

Your project should make appropriate use of Java collections.

Depending on your application, you may use:

- `List`
- `Set`
- `Map`

You are also encouraged to use Java features studied during the course where they make sense, such as:

- `Optional`
- Streams
- Lambda expressions
- Enums
- `LocalDate` / `LocalDateTime`
- `Generics`

Do not use a feature simply because it exists.

The important point is to choose the appropriate tool for the problem.

---

9. Application Flow

Your application should have a clear execution flow.

For example:

Application starts
↓
Load data from file
↓
Create domain objects
↓
Execute business operations
↓
Display results
↓
Handle possible errors

The exact interaction model is up to you.

A simple console application is completely acceptable.

You **do not need to create a graphical interface or web application**.

The objective of this project is to evaluate your understanding of Java and OOP.

---

10. Code Quality

Your code should be:

- Readable
- Organized
- Consistent
- Properly named
- Easy to understand
- Free from unnecessary duplication

Use appropriate:

- Package structure
- Class names
- Method names
- Variable names
- Comments where necessary

Comments should explain **why** something is done when the reason is not obvious, rather than simply repeating what the code does.

---

11. Project Structure

A possible structure could be:

```
src/
├── main/
│   └── java/
│       └── com.example.project/
│           ├── model/ (your entities /model classes)
│           ├── service/ (your classes handling business logic for your entities)
│           ├── exception/
│           ├── utils/ (any helper class required to run your application)
│           └── Application.java
│
├── test/
│   └── java/
│       └── com.example.project/
│           └── ...
```

```
│
└── resources/
    └── data.csv
```

This is only an example.

You may organize your project differently if your structure is coherent.

---

12. Deliverables

You must submit you project on a github repository

1. Source code

Your complete Java project. This includes src and test folders.

The project must compile and run.

The source code should be delivered on a GitHub repository before the due date. Any change after that date will not be taken into account.

2. Data file

The file used to initialize your application.

For example: data.json or data.csv

3. README

Your project must contain a `README.md` explaining:

- The purpose of your application
- How to run it
- The main features
- The main classes
- **Some technical decisions (between 2 and 5) that you took and why**
- Where inheritance is used
- Where polymorphism is used
- Where the abstract class/interface is used
- Which custom exceptions you created
- How the application loads its data
- How you applied SOLID principles
- How to run the tests

---

13. Final Notes

The goal of this project is **not to build the biggest application possible**.

A small, well-designed application is better than a large application with poor architecture.

You must be able to explain and justify the code that you submit.

Focus on:

Good object-oriented design + meaningful business logic + clean Java code + tests.

You are free to be creative with your project subject.

Before going into coding, I recommend you think about your project's subject : classes, links between classes, business logic & features, use cases, errors, tests.